

PR_10.2 Dani Gayol Rodríguez

PR_10.2 Dani Gayol Rodríguez.....	1
Apartado A.....	1
Escribe los comandos necesarios para realizar las siguientes operaciones DML sobre la colección zips:	2
1.) Crea una entrada con los siguientes datos:	2
2.) Crea una entrada con los datos del código postal donde vives (si es el mismo código postal, crea uno diferente), con el valor de state a España.....	3
3.) Modifica la población de tu código postal a 1.000.000.....	4
4.) Incrementa la población de todos los documentos de España en 666 personas.	4
5.) Añade un campo prov a ambos documentos con valor Alicante.	5
6.) Renombra en los documentos de la provincia de Alicante el atributo prov por provincia	6
7.) Elimina las coordenadas del zip 03206.....	7
8.) Elimina tu entrada.	8
Apartado B.....	8
Escribe los comandos necesarios para realizar las siguientes operaciones sobre la colección sample_restaurants.restaurants:	9
1.) Insertar un nuevo restaurante de comida española con tu nombre, pero sin ninguna calificación.	9
2.) Añade una calificación al restaurante	10
3.) Cambiar el nombre del restaurante por Severo manjar.	11
4.) Modifica el número del edificio de la dirección.	12
5.) Modifica el tipo de cocina para que sea Mediterránea.	13
6.) Añade una segunda calificación.	13
7.) Modifica las coordenadas de la ubicación para que coincidan con las de tu ciudad.....	14
8.) Modifica el barrio con el barrio donde vives.	15
9.) Añade una tercera calificación que sea excelente.....	16

10.) Modifica la calle y el código postal mediante una única operación.	17
11.) Elimina la calificación más antigua.	18
12.) Añade un nuevo campo horario con el valor que consideres.	19
13. Incrementa todas las calificaciones en un punto.	20
Apartado C	21
Escribe los comandos necesarios para realizar las siguientes operaciones de arrays sobre la colección zips:	22
1.) Modifica los documentos de España y añade un atributo tags que contenga un array vacío.....	22
2.) Añade a todos los documentos que tenga el atributo tags el valor life dentro del array.	22
3.) Modifica todos los documentos de la provincia de Alicante y añade al atributo tags el valor sun.	23
4.) Añade al final de tags el atributo food a todas las ciudades de España.	24
5.) Mediante una operación, añade un nuevo atributo tourism con los valores sun y food a todos los documentos de España.	25
6.) Añade en la segunda posición de todos los documentos de España el valor drfleming en la propiedad tags.	26
7.) Modifica el valor de sun de tags de tu código postal y sustitúyelo por house. ..	27
8.) Elimina el valor house de tags de tu código postal.....	28
9.) Elimina el último valor de tags.	29
10.) Elimina el primer valor de tourism.	30
Apartado D	31
Utilizando las herramientas de administración de MongoDB (las cuales puedes utilizarlas desde dentro de Docker o bien descargar las MongoTools), se pide:	32
1.) Exporta los datos de la colección zips en drfleming_zips.json	32
2.) Importa la colección recién exportada dentro de la base de datos pruebas en la colección drfleming_zips.	32
3.) Realiza una consulta y comprueba que los datos se han importado correctamente.....	33
4.) Exporta la base de datos de sample_trainings y muestra el resultado.	34
5.) Importa la base de datos recién exportada en drfleming_trainings	35
6.) Realiza una consulta y comprueba que los datos se han importado correctamente.....	36

7.) Importa el archivo clientes.csv dentro de la base de datos pruebas en la colección clientes.....	37
8.) Realiza una consulta y comprueba que los datos se han importado correctamente.....	37
9.) Importa el archivo poblacion.tsv dentro de la base de datos pruebas en la colección poblacion.....	38
10.) Realiza una consulta y comprueba que los datos se han importado correctamente.....	39
Apartado E.....	40
Escribe los comandos necesarios para realizar las siguientes consultas agregadas sobre la colección sample_supplies.sales:	41
1.) Cantidad total de ventas registradas.....	41
2.) Cantidad total de ventas por método de compra (purchaseMethod).....	41
3.) Investiga el operador \$unwind y calcula el ingreso total de cada producto (name dentro de items)	41
4.) Calcular el promedio de elementos por venta.....	42
5.) A partir de todos los elementos, calcula el ingreso total por ventas, teniendo en cuenta el precio y la cantidad de cada elemento.	43
6.) Encontrar los 3 productos más vendidos (por cantidad de unidades vendidas).	44
7.) Averiguar cuantas ventas se han realizado del producto notepad.....	45
8.) Averiguar cuantos notepad se han vendido y cuanto se ha recaudado por sus ventas.....	45
9.) Averiguar los tres productos con un precio medio más alto.	46
10.) Averiguar la cantidad de hombres y mujeres que han realizado compras, así como sus edades medias.	47
11.) Recuperar el nombre y el email de los 5 clientes que más han gastado.....	48
12.) Para cada email, mostrar la cantidad de ventas realizadas, así como la cantidad de productos adquiridos.....	49
13.) Recupera cual es el producto que más se ha vendido en London.	50
14.) Obtener el nombre y el email del cliente, y el total gastado, pero solo para aquellos clientes que han gastado más de 1000\$.	51
15.) Averiguar cuáles han sido los tres años que más ventas se han realizado.	52
Apartado F.....	53

Escribe los comandos necesarios para realizar las siguientes consultas agregadas sobre la colección <code>sample_restaurants.restaurants</code> :	54
1.) Cantidad total de restaurantes.	54
2.) Cantidad de restaurantes por barrio (propiedad <code>borough</code>).	54
3.) Recupera los cinco barrios con mayor cantidad de restaurantes.	55
4.) Investiga el operador <code>\$unwind</code> , y recupera la calificación media del restaurante <code>Palm Restauran</code> .	56
5.) Recupera la fecha de la última calificación de cada restaurante.	57
6.) Encuentra todos los restaurantes que al menos tienen una calificación de A.	58
7.) Mediante <code>\$unwind</code> , cuenta cuántas calificaciones de tipo A, B, C, etc.... existen en total en todos los restaurantes.	59
8.) Mostrar los restaurantes que tienen más de tres calificaciones.	60
9.) Mostrar los restaurantes que tienen más de cuatro calificaciones y que hagan cocina <code>American</code> .	61
10.) Encontrar el restaurante con la calificación A más alta (mayor score).	62
11.) Calcular la diferencia entre el score más alto y bajo para cada restaurante que tenga al menos dos grades.	63
12.) Obtener el nombre del restaurante, su dirección (<code>street</code> y <code>zipcode</code>) y la última fecha de inspección (<code>grade.date</code>) para los restaurantes del Bronx.	64
13.) Encontrar los 3 barrios tienen la mayor variedad de tipos de cocina.	65
Apartado G	66
Importa el fichero <code>mongo_cities1000.json</code> en <code>pruebas.cities</code> . Sobre dichos datos, resuelve las siguientes consultas:	67
1.) Calcula el número total de ciudades.	67
2.) Recupera todos los datos de la ciudad cuyo nombre es <code>Oviedo</code> .	67
3.) Recupera únicamente la población de <code>Vergel</code> .	68
4.) Calcula el número de ciudades de España.	68
5.) Datos de las ciudades españolas con más de un millón de habitantes.	68
6.) Recupera la cantidad de ciudades que hay entre <code>Andorra (AD)</code> y España.	69
7.) Listado con el nombre y la población de las 10 ciudades más pobladas.	69
8.) Nombre de las distintas zonas horarias en España.	70
9.) Ciudades españolas que su zona horaria no sea <code>Europe/Madrid</code> .	70
10.) Mostrar las ciudades españolas que comiencen por <code>Ben</code> .	71

11.) Devolver las ciudades que su zona horaria sea Atlantic/Canary o Africa/Ceuta.	72
12.) Obtén el nombre y la población de las tres ciudades europeas más pobladas	73
13.) Calcula la cantidad de ciudades españolas cuyas coordenadas de longitud estén comprendidas entre -0.1 y 0.1.....	74
14.) Mostrar los 3 países con mayor número de ciudades.	74
15.) Devolver los 3 países con mayor número de habitantes.....	75
16.) Calcular y devolver la población total de España, Francia e Italia, en orden descendente.	76
17.) Calcular la latitud y longitud media para cada zona horaria europea.	77
18.) Devolver los países en la posición 5ª y 6ª si ordenamos su población total en orden descendente.	78
19.) Calcular las zonas horarias con mayor y menor número de habitantes para España y Rusia. Añade también un campo extra que indique cuántas zonas horarias tiene cada país.	79
20.) Calcular qué porcentaje de la población representa la ciudad más poblada de España.....	80
21.) Utilizando unwind, devuelve los pares únicos (país y timezone) con un campo que indique la suma total de población del país.	81
22.) Averigua qué nombres de ciudades se repiten entre distintos países.	82

Apartado A

Escribe los comandos necesarios para realizar las siguientes operaciones DML sobre la colección zips:

```
>_MONGOSH  
  
> use sample_training  
< switched to db sample_training  
Atlas atlas-lkwc66-shard-0 [primary] sample_training> |
```

1.) Crea una entrada con los siguientes datos:

```
> db.zips.insertOne({  
  city: "OVIEDO",  
  zip: "33001",  
  loc: { x: 43.3625025223589, y: -5.843830860844533 },  
  pop: 226872,  
  state: "España"  
})  
< {  
  acknowledged: true,  
  insertedId: ObjectId('699d59c78e6655123a3f78d1')  
}
```

 {city: "OVIEDO"}

 ADD DATA ▾

 UPDATE

 DELETE

 EXPORT CODE



```
_id: ObjectId('5c8eccc1caal87d17ca6fd5b')
city: "OVIEDO"
zip: "32765"
loc: Object
pop: 19519
state: "FL"
```

```
_id: ObjectId('699d59c78e6655123a3f78d1')
city: "OVIEDO"
zip: "33001"
loc: Object
pop: 226872
state: "España"
```

2.) Crea una entrada con los datos del código postal donde vives (si es el mismo código postal, crea uno diferente), con el valor de state a España.

```
> db.zips.insertOne({
  city: "GIJON",
  zip: "33213",
  loc: { x: 43.5294, y: -5.6772 },
  pop: 269894,
  state: "España"
})
< {
  acknowledged: true,
  insertedId: ObjectId('699d5ad58e6655123a3f78d2')
}
```

 {city: "GIJON"}

 ADD DATA

 UPDATE

 DELETE

 EXPORT CODE



```
_id: ObjectId('699d5ad58e6655123a3f78d2')
city: "GIJON"
zip: "33213"
loc: Object
pop: 269894
state: "España"
```

3.) Modifica la población de tu código postal a 1.000.000.

```
> db.zips.updateOne(
  { zip: "33213" },
  { $set: { pop: 1000000 } }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

 {city: "GIJON"}

 ADD DATA

 UPDATE

 DELETE

 EXPORT CODE





```
_id: ObjectId('699d5ad58e6655123a3f78d2')
city: "GIJON"
zip: "33213"
loc: Object
pop: 1000000
state: "España"
```


4.) Incrementa la población de todos los documentos de España en 666 personas.

```
> db.zips.updateMany(  
  { state: "España" },  
  { $inc: { pop: 666 } }  
)  
< {  
  acknowledged: true,  
  insertedId: null,  
  matchedCount: 2,  
  modifiedCount: 2,  
  upsertedCount: 0  
}
```

🕒 {state: "España"}

+ ADD DATA ▾

✎ UPDATE

🗑 DELETE

📄 EXPORT CODE



```
_id: ObjectId('699d59c78e6655123a3f78d1')  
city: "OVIEDO"  
zip: "33001"  
▸ loc: Object  
pop: 227538  
state: "España"
```

```
_id: ObjectId('699d5ad58e6655123a3f78d2')  
city: "GIJON"  
zip: "33213"  
▸ loc: Object  
pop: 1000666  
state: "España"
```

5.) Añade un campo prov a ambos documentos con valor Alicante.

```
> db.zips.updateMany(
  { state: "España" },
  { $set: { prov: "Alicante" } }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 2,
  modifiedCount: 2,
  upsertedCount: 0
}
```

🕒 {state: "España"}

+ ADD DATA ▾

✎ UPDATE

🗑 DELETE

📄 EXPORT CODE



```
_id: ObjectId('699d59c78e6655123a3f78d1')
city: "OVIEDO"
zip: "33001"
▸ loc: Object
pop: 227538
state: "España"
prov: "Alicante"
```

```
_id: ObjectId('699d5ad58e6655123a3f78d2')
city: "GIJON"
zip: "33213"
▸ loc: Object
pop: 1000666
state: "España"
prov: "Alicante"
```

6.) Renombra en los documentos de la provincia de Alicante el atributo prov por provincia

```
> db.zips.updateMany(
  { prov: "Alicante" },
  { $rename: { "prov": "provincia" } }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 2,
  modifiedCount: 2,
  upsertedCount: 0
}
```

🕒 {state: "España"}

+ ADD DATA ▾

✎ UPDATE

🗑 DELETE

📄 EXPORT CODE



```
_id: ObjectId('699d59c78e6655123a3f78d1')
city: "OVIEDO"
zip: "33001"
▸ loc: Object
pop: 227538
state: "España"
provincia: "Alicante"
```

```
_id: ObjectId('699d5ad58e6655123a3f78d2')
city: "GIJON"
zip: "33213"
▸ loc: Object
pop: 1000666
state: "España"
provincia: "Alicante"
```

7.) Elimina las coordenadas del zip 03206.

```
> db.zips.updateOne(
  { zip: "03206" },
  { $unset: { loc: "" } }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 0,
  modifiedCount: 0,
  upsertedCount: 0
}
```

No existe ningún código postal con ese zip

8.) Elimina tu entrada.

```
> db.zips.deleteOne({ zip: "33213" })
< {
  acknowledged: true,
  deletedCount: 1
}
```

🕒 {zip: "33213"}

[ADD DATA](#) [UPDATE](#) [DELETE](#) [EXPORT CODE](#) ?

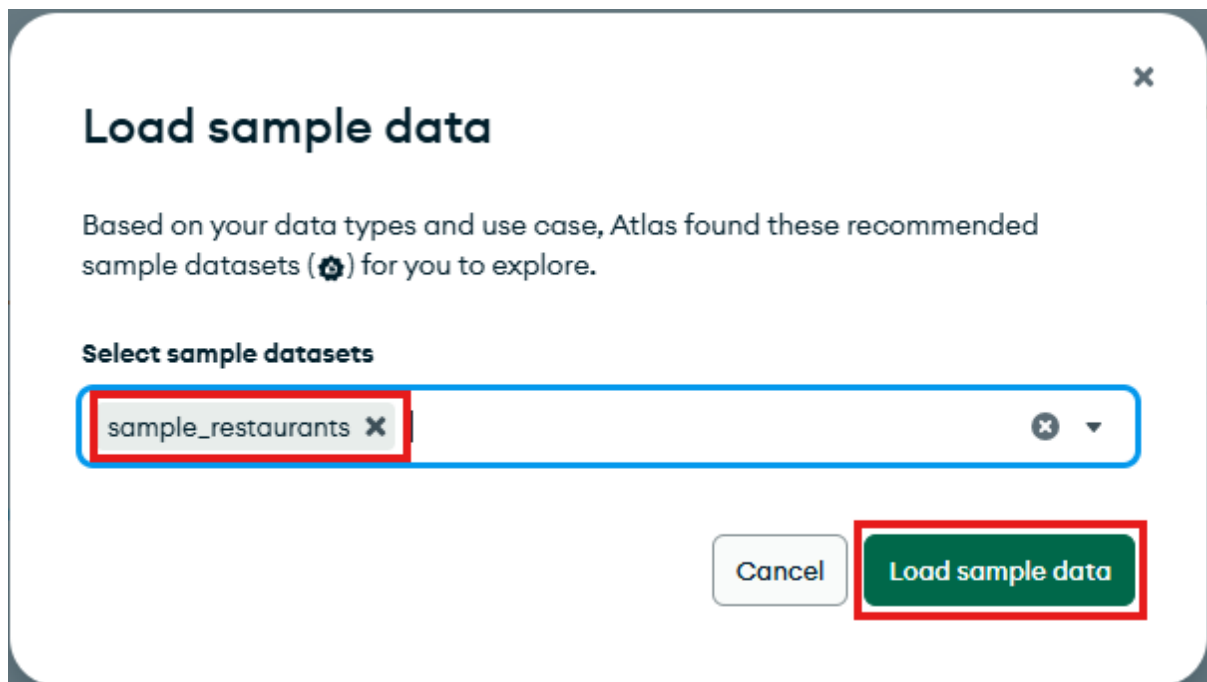


No results

Try modifying your query to get results.

Apartado B

Escribe los comandos necesarios para realizar las siguientes operaciones sobre la colección `sample_restaurants.restaurants`:



```
> use sample_restaurants
< switched to db sample_restaurants
```

1.) Insertar un nuevo restaurante de comida española con tu nombre, pero sin ninguna calificación.

```
> db.restaurants.insertOne({
  name: "Restaurante DaniGayol",
  cuisine: "Spanish",
  borough: "Centro",
  address: {
    building: "12",
    street: "Calle Mayor",
    zipcode: "33213",
    coord: [43.4907, 5.3452]
  },
  grades: []
})
< {
  acknowledged: true,
  insertedId: ObjectId('699d60de8e6655123a3f78d3')
}
```

🕒 {name: "Restaurante DaniGayol"}

+ ADD DATA ▾

✎ UPDATE

🗑 DELETE

📄 EXPORT CODE



```
_id: ObjectId('699d60de8e6655123a3f78d3')
name: "Restaurante DaniGayol"
cuisine: "Spanish"
borough: "Centro"
▼ address: Object
  building: "12"
  street: "Calle Mayor"
  zipcode: "33213"
  ▼ coord: Array (2)
    0: 43.4907
    1: 5.3452
  ► grades: Array (empty)
```

2.) Añade una calificación al restaurante

```
> db.restaurants.updateOne(
  { name: "Restaurante DaniGayol" },
  {
    $push: {
      grades: {
        date: new Date(),
        grade: "A",
        score: 8
      }
    }
  }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

🕒 ▼ {name: "Restaurante DaniGayol"}

➕ ADD DATA ▼

✎ UPDATE

🗑 DELETE

📄 EXPORT CODE



```
_id: ObjectId('699d60de8e6655123a3f78d3')
name: "Restaurante DaniGayol"
cuisine: "Spanish"
borough: "Centro"
▶ address: Object
▼ grades: Array (1)
  ▼ 0: Object
    date: 2026-02-24T08:29:01.272+00:00
    grade: "A"
    score: 8
```

3.) Cambiar el nombre del restaurante por Severo manjar.

```
> db.restaurants.updateOne(
  { name: "Restaurante DaniGayol" },
  { $set: { name: "Severo manjar" } }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

🕒 {name: "Severo manjar"}

➕ ADD DATA ▼

✎ UPDATE

🗑 DELETE

📄 EXPORT CODE



```
_id: ObjectId('699d60de8e6655123a3f78d3')
name: "Severo manjar"
cuisine: "Spanish"
borough: "Centro"
▶ address: Object
▶ grades: Array (1)
```

4.) Modifica el número del edificio de la dirección.

```
> db.restaurants.updateOne(
  { name: "Severo manjar" },
  { $set: { "address.building": "99" } }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```


 {name: "Severo manjar"}

 ADD DATA

 UPDATE

 DELETE

 EXPORT CODE



```
_id: ObjectId('699d60de8e6655123a3f78d3')
name: "Severo manjar"
cuisine: "Spanish"
borough: "Centro"
▼ address: Object
  building: "99"
  street: "Calle Mayor"
  zipcode: "33213"
  ▶ coord: Array (2)
  ▶ grades: Array (1)
```

5.) Modifica el tipo de cocina para que sea Mediterránea.

```
> db.restaurants.updateOne(
  { name: "Severo manjar" },
  { $set: { cuisine: "Mediterranea" } }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

 {name: "Severo manjar"}

 ADD DATA

 UPDATE

 DELETE

 EXPORT CODE



```
_id: ObjectId('699d60de8e6655123a3f78d3')
name: "Severo manjar"
cuisine: "Mediterranea"
borough: "Centro"
▶ address: Object
▶ grades: Array (1)
```

6.) Añade una segunda calificación.

```
> db.restaurants.updateOne(
  { name: "Severo manjar" },
  {
    $push: {
      grades: {
        date: new Date(),
        grade: "A",
        score: 9
      }
    }
  }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

🕒 ▼ {name: "Severo manjar"}

➕ ADD DATA ▼

✎ UPDATE

🗑 DELETE

📄 EXPORT CODE



```
_id: ObjectId('699d60de8e6655123a3f78d3')
name: "Severo manjar"
cuisine: "Mediterranea"
borough: "Centro"
▶ address: Object
▼ grades: Array (2)
  ▶ 0: Object
  ▼ 1: Object
    date: 2026-02-24T08:32:22.524+00:00
    grade: "A"
    score: 9
```

7.) Modifica las coordenadas de la ubicación para que coincidan con las de tu ciudad.

```
> db.restaurants.updateOne(
  { name: "Severo manjar" },
  { $set: { "address.coord": [43.3625025223589, -5.843830860844533] } }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

🕒 {name: "Severo manjar"}

➕ ADD DATA ▾

✎ UPDATE

🗑 DELETE

📄 EXPORT CODE



```
_id: ObjectId('699d60de8e6655123a3f78d3')
name: "Severo manjar"
cuisine: "Mediterranea"
borough: "Centro"
▼ address: Object
  building: "99"
  street: "Calle Mayor"
  zipcode: "33213"
  ▼ coord: Array (2)
    0: 43.3625025223589
    1: -5.843830860844533
  ► grades: Array (2)
```

8.) Modifica el barrio con el barrio donde vives.

```
> db.restaurants.updateOne(
  { name: "Severo manjar" },
  { $set: { borough: "La Calzada" } }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

🕒 {name: "Severo manjar"}

+ ADD DATA ▾

✎ UPDATE

🗑 DELETE

📄 EXPORT CODE



```
_id: ObjectId('699d60de8e6655123a3f78d3')
name: "Severo manjar"
cuisine: "Mediterranea"
borough: "La Calzada"
▸ address: Object
▸ grades: Array (2)
```

9.) Añade una tercera calificación que sea excelente.

```
> db.restaurants.updateOne(
  { name: "Severo manjar" },
  {
    $push: {
      grades: {
        date: new Date(),
        grade: "A",
        score: 10
      }
    }
  }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

🕒 ▼ {name: "Severo manjar"}

➕ ADD DATA ▼

✎ UPDATE

🗑 DELETE

📄 EXPORT CODE



```
_id: ObjectId('699d60de8e6655123a3f78d3')
name : "Severo manjar"
cuisine : "Mediterranea"
borough : "La Calzada"
▸ address : Object
▼ grades : Array (3)
  ▸ 0: Object
  ▸ 1: Object
  ▼ 2: Object
    date : 2026-02-24T08:35:11.081+00:00
    grade : "A"
    score : 10
```

10.) Modifica la calle y el código postal mediante una única operación.

```
> db.restaurants.updateOne(
  { name: "Severo manjar" },
  {
    $set: {
      "address.street": "Calle Mayor 2",
      "address.zipcode": "33208"
    }
  }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

🕒 ▼ {name: "Severo manjar"}

➕ ADD DATA ▼

✎ UPDATE

🗑 DELETE

📄 EXPORT CODE



```
_id: ObjectId('699d60de8e6655123a3f78d3')
name: "Severo manjar"
cuisine: "Mediterranea"
borough: "La Calzada"
▼ address: Object
  building: "99"
  street: "Calle Mayor 2"
  zipcode: "33208"
  ▶ coord: Array (2)
  ▶ grades: Array (3)
```

11.) Elimina la calificación más antigua.

```
> db.restaurants.updateOne(
  { name: "Severo manjar" },
  { $pop: { grades: -1 } }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

🕒 ▼ {name: "Severo manjar"}

➕ ADD DATA ▼

✎ UPDATE

🗑 DELETE

📄 EXPORT CODE



```
_id: ObjectId('699d60de8e6655123a3f78d3')
name : "Severo manjar"
cuisine : "Mediterranea"
borough : "La Calzada"
▶ address : Object
▼ grades : Array (2)
  ▼ 0: Object
    date : 2026-02-24T08:32:22.524+00:00
    grade : "A"
    score : 9
  ▼ 1: Object
    date : 2026-02-24T08:35:11.081+00:00
    grade : "A"
    score : 10
```

12.) Añade un nuevo campo horario con el valor que consideres.

```
> db.restaurants.updateOne(
  { name: "Severo manjar" },
  { $set: { horario: "Lunes a Domingo 13:00 - 23:00" } }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

🕒 ▼ {name: "Severo manjar"}

+ ADD DATA ▼

✎ UPDATE

🗑 DELETE

📄 EXPORT CODE



```
_id: ObjectId('699d60de8e6655123a3f78d3')
name : "Severo manjar"
cuisine : "Mediterranea"
borough : "La Calzada"
▶ address : Object
▶ grades : Array (2)
horario : "Lunes a Domingo 13:00 - 23:00"
```


13. Incrementa todas las calificaciones en un punto.

```
> db.restaurants.updateOne(
  { name: "Severo manjar" },
  {
    $inc: {
      "grades.$.score": 1
    }
  }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

🕒 ▼ {name: "Severo manjar"}

+ ADD DATA ▼

✎ UPDATE

🗑 DELETE

📄 EXPORT CODE



```
_id: ObjectId('699d60de8e6655123a3f78d3')
name: "Severo manjar"
cuisine: "Mediterranea"
borough: "La Calzada"
▶ address: Object
▼ grades: Array (2)
  ▼ 0: Object
    date: 2026-02-24T08:32:22.524+00:00
    grade: "A"
    score: 10
  ▼ 1: Object
    date: 2026-02-24T08:35:11.081+00:00
    grade: "A"
    score: 11
horario: "Lunes a Domingo 13:00 - 23:00"
```

Apartado C

Escribe los comandos necesarios para realizar las siguientes operaciones de arrays sobre la colección zips:

```
> use sample_training  
< switched to db sample_training
```

1.) Modifica los documentos de España y añade un atributo tags que contenga un array vacío.

```
> db.zips.updateMany(  
  { state: "España" },  
  { $set: { tags: [] } }  
)  
< {  
  acknowledged: true,  
  insertedId: null,  
  matchedCount: 1,  
  modifiedCount: 1,  
  upsertedCount: 0  
}
```

🕒 ▼ {state: "España"}

➕ ADD DATA ▼

✎ UPDATE

🗑 DELETE

📄 EXPORT CODE



```
_id: ObjectId('699d59c78e6655123a3f78d1')  
city: "OVIEDO"  
zip: "33001"  
▸ loc: Object  
pop: 227538  
state: "España"  
provincia: "Alicante"  
▼ tags: Array (empty)
```

2.) Añade a todos los documentos que tenga el atributo tags el valor life dentro del array.

```
> db.zips.updateMany(
  { tags: { $exists: true } },
  { $push: { tags: "life" } }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

🕒 {state: "España"}

➕ ADD DATA ▾

✎ UPDATE

🗑 DELETE

📄 EXPORT CODE



```
_id: ObjectId('699d59c78e6655123a3f78d1')
city: "OVIEDO"
zip: "33001"
▸ loc: Object
  pop: 227538
  state: "España"
  provincia: "Alicante"
▾ tags: Array (1)
  0: "life"
```

3.) Modifica todos los documentos de la provincia de Alicante y añade al atributo tags el valor sun.

```
> db.zips.updateMany(  
  { provincia: "Alicante" },  
  { $push: { tags: "sun" } }  
)  
< {  
  acknowledged: true,  
  insertedId: null,  
  matchedCount: 1,  
  modifiedCount: 1,  
  upsertedCount: 0  
}
```

🕒 {state: "España"}

+ ADD DATA ▾

✎ UPDATE

🗑 DELETE

📄 EXPORT CODE



```
_id: ObjectId('699d59c78e6655123a3f78d1')  
city: "OVIEDO"  
zip: "33001"  
▸ loc: Object  
pop: 227538  
state: "España"  
provincia: "Alicante"  
▾ tags: Array (2)  
  0: "life"  
  1: "sun"
```

4.) Añade al final de tags el atributo food a todas las ciudades de España.

```
> db.zips.updateMany(
  { state: "España" },
  { $push: { tags: "food" } }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

🕒 {state: "España"}

➕ ADD DATA ▾

✎ UPDATE

🗑 DELETE

📄 EXPORT CODE



```
_id: ObjectId('699d59c78e6655123a3f78d1')
city: "OVIEDO"
zip: "33001"
▸ loc: Object
pop: 227538
state: "España"
provincia: "Alicante"
▾ tags: Array (3)
  0: "life"
  1: "sun"
  2: "food"
```

5.) Mediante una operación, añade un nuevo atributo tourism con los valores sun y food a todos los documentos de España.

```
> db.zips.updateMany(
  { state: "España" },
  { $set: { tourism: ["sun", "food"] } }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

🕒 {state: "España"}

➕ ADD DATA ▾

✎ UPDATE

🗑 DELETE

📄 EXPORT CODE



```
_id: ObjectId('699d59c78e6655123a3f78d1')
city: "OVIEDO"
zip: "33001"
▸ loc: Object
pop: 227538
state: "España"
provincia: "Alicante"
▸ tags: Array (3)
▼ tourism: Array (2)
  0: "sun"
  1: "food"
```

6.) Añade en la segunda posición de todos los documentos de España el valor drfleming en la propiedad tags.

```
> db.zips.updateMany(
  { state: "España" },
  {
    $push: {
      tags: {
        $each: ["drfleming"],
        $position: 1
      }
    }
  }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

🕒 {state: "España"}

+ ADD DATA ▾

✎ UPDATE

🗑 DELETE

📄 EXPORT CODE



```
_id: ObjectId('699d59c78e6655123a3f78d1')
city: "OVIEDO"
zip: "33001"
▸ loc: Object
pop: 227538
state: "España"
provincia: "Alicante"
▾ tags: Array (4)
  0: "life"
  1: "drfleming"
  2: "sun"
  3: "food"
▸ tourism: Array (2)
```

7.) Modifica el valor de sun de tags de tu código postal y sustitúyelo por house.

```
> db.zip.updateOne(
  { zip: "33001", tags: "sun" },
  { $set: { "tags.$": "house" } }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

🕒 {state: "España"}

+ ADD DATA ▾

✎ UPDATE

🗑 DELETE

📄 EXPORT CODE



```
_id: ObjectId('699d59c78e6655123a3f78d1')
city: "OVIEDO"
zip: "33001"
▸ loc: Object
pop: 227538
state: "España"
provincia: "Alicante"
▾ tags: Array (4)
  0: "life"
  1: "drfleming"
  2: "house"
  3: "food"
▸ tourism: Array (2)
```


8.) Elimina el valor house de tags de tu código postal.

```
> db.zips.updateOne(
  { zip: "33001" },
  { $pull: { tags: "house" } }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

🕒 {state: "España"}

+ ADD DATA ▾

✎ UPDATE

🗑 DELETE

📄 EXPORT CODE



```
_id: ObjectId('699d59c78e6655123a3f78d1')
city: "OVIEDO"
zip: "33001"
▸ loc: Object
pop: 227538
state: "España"
provincia: "Alicante"
▾ tags: Array (3)
  0: "life"
  1: "drfleming"
  2: "food"
▸ tourism: Array (2)
```

9.) Elimina el último valor de tags.

```
> db.zips.updateMany(
  { state: "España" },
  { $pop: { tags: 1 } }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

🕒 {state: "España"}

➕ ADD DATA ▾

✎ UPDATE

🗑 DELETE

📄 EXPORT CODE



```
_id: ObjectId('699d59c78e6655123a3f78d1')
city: "OVIEDO"
zip: "33001"
▸ loc: Object
  pop: 227538
  state: "España"
  provincia: "Alicante"
▾ tags: Array (2)
  0: "life"
  1: "drfleming"
▸ tourism: Array (2)
```

10.) Elimina el primer valor de tourism.

```
> db.zips.updateMany(  
  { state: "España" },  
  { $pop: { tourism: -1 } }  
)  
< {  
  acknowledged: true,  
  insertedId: null,  
  matchedCount: 1,  
  modifiedCount: 1,  
  upsertedCount: 0  
}
```

🕒 {state: "España"}

➕ ADD DATA ▾

✎ UPDATE

🗑 DELETE

📄 EXPORT CODE



```
_id: ObjectId('699d59c78e6655123a3f78d1')  
city: "OVIEDO"  
zip: "33001"  
▸ loc: Object  
pop: 227538  
state: "España"  
provincia: "Alicante"  
▸ tags: Array (2)  
▼ tourism: Array (1)  
  0: "food"
```

Apartado D

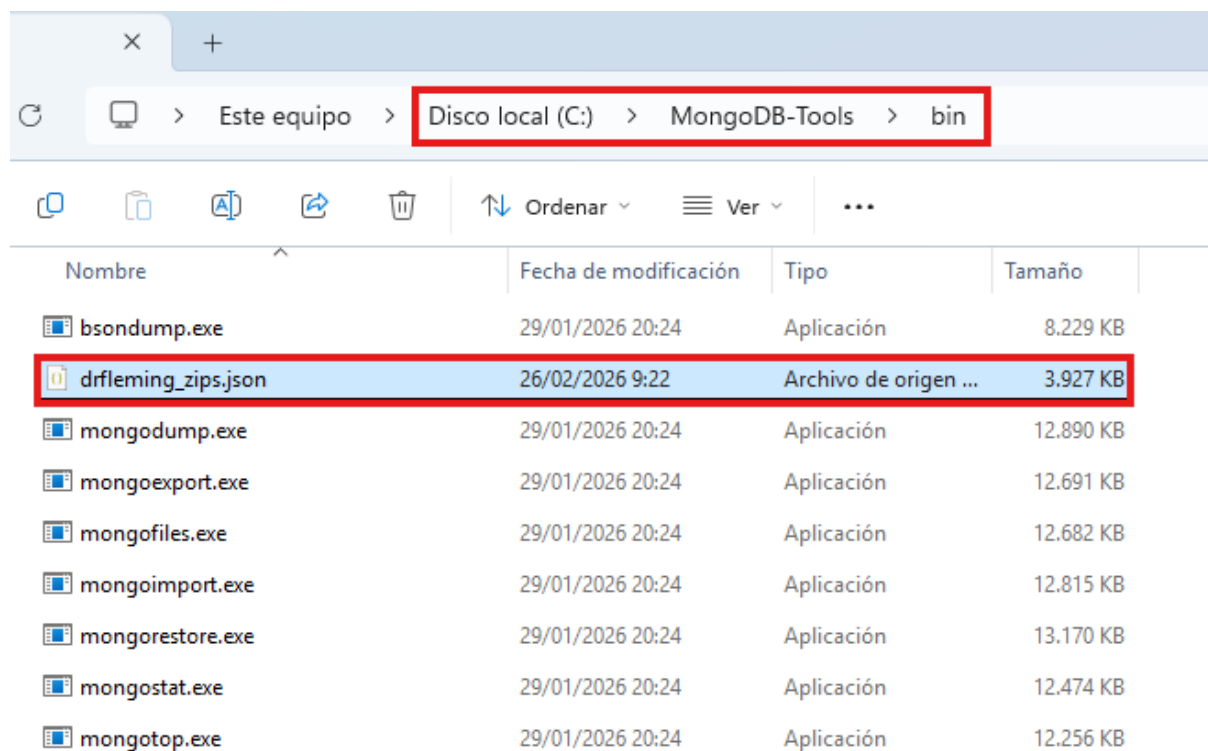
Utilizando las herramientas de administración de MongoDB (las cuales puedes utilizarlas desde dentro de Docker o bien descargar las MongoTools), se pide:

1.) Exporta los datos de la colección zips en drfleming_zips.json

```
C:\Users\Mañana>cd "C:\MongoDB-Tools\bin"

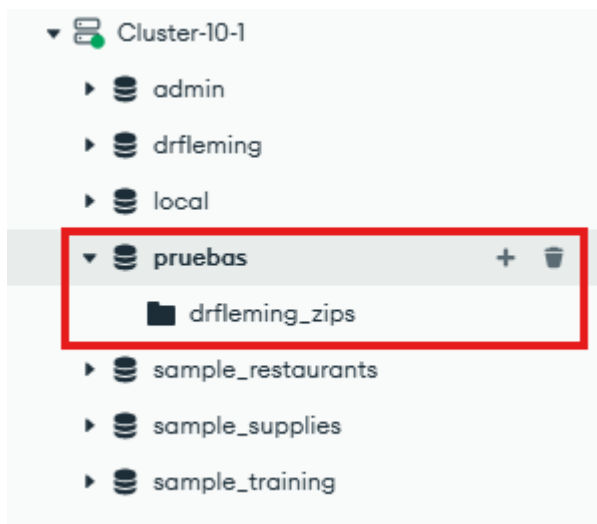
C:\MongoDB-Tools\bin>mongoexport --uri="mongodb+srv://admin:admin@cluster-10-1.fegquti.mongodb.net/" --db sample_trainin
g --collection zips --out drfleming_zips.json --jsonArray
2026-02-26T09:21:33.260+0100 connected to: mongodb+srv://[**REDACTED**]@cluster-10-1.fegquti.mongodb.net/
2026-02-26T09:21:34.337+0100 [.....] sample_training.zips 0/29471 (0.0%)

2026-02-26T09:22:39.336+0100 [#####.....] sample_training.zips 16000/29471 (54.3%)
2026-02-26T09:22:39.597+0100 [#####.....] sample_training.zips 29471/29471 (100.0%)
2026-02-26T09:22:39.597+0100 exported 29471 records
```



2.) Importa la colección recién exportada dentro de la base de datos pruebas en la colección drfleming_zips.

```
C:\MongoDB-Tools\bin>mongoimport --uri="mongodb+srv://admin:admin@cluster-10-1.fegquti.mongodb.net/" --db pruebas --collection drfleming_zips --file drfleming_zips.json --jsonArray
connected to: mongodb+srv://[**REDACTED**]@cluster-10-1.fegquti.mongodb.net/
2026-02-26T09:27:14.834+0100 [#.....] pruebas.drfleming_zips 272KB/3.83MB (6.9%)
2026-02-26T09:27:17.835+0100 [##.....] pruebas.drfleming_zips 407KB/3.83MB (10.4%)
2026-02-26T09:27:23.835+0100 [###.....] pruebas.drfleming_zips 407KB/3.83MB (10.4%)
2026-02-26T09:27:26.835+0100 [####.....] pruebas.drfleming_zips 541KB/3.83MB (13.8%)
2026-02-26T09:27:29.835+0100 [#####.....] pruebas.drfleming_zips 941KB/3.83MB (24.0%)
2026-02-26T09:27:32.834+0100 [#####.....] pruebas.drfleming_zips 1.31MB/3.83MB (34.1%)
2026-02-26T09:27:35.835+0100 [#####.....] pruebas.drfleming_zips 1.57MB/3.83MB (40.9%)
2026-02-26T09:27:38.834+0100 [#####.....] pruebas.drfleming_zips 1.83MB/3.83MB (47.6%)
2026-02-26T09:27:41.835+0100 [#####.....] pruebas.drfleming_zips 2.35MB/3.83MB (61.2%)
2026-02-26T09:27:44.835+0100 [#####.....] pruebas.drfleming_zips 2.61MB/3.83MB (68.0%)
2026-02-26T09:27:47.835+0100 [#####.....] pruebas.drfleming_zips 2.61MB/3.83MB (68.0%)
2026-02-26T09:27:50.835+0100 [#####.....] pruebas.drfleming_zips 2.87MB/3.83MB (74.8%)
2026-02-26T09:27:53.834+0100 [#####.....] pruebas.drfleming_zips 3.00MB/3.83MB (78.2%)
2026-02-26T09:27:56.835+0100 [#####.....] pruebas.drfleming_zips 3.00MB/3.83MB (78.2%)
2026-02-26T09:27:59.834+0100 [#####.....] pruebas.drfleming_zips 3.26MB/3.83MB (85.0%)
2026-02-26T09:28:02.835+0100 [#####.....] pruebas.drfleming_zips 3.39MB/3.83MB (88.4%)
2026-02-26T09:28:05.835+0100 [#####.....] pruebas.drfleming_zips 3.39MB/3.83MB (88.4%)
2026-02-26T09:28:08.835+0100 [#####.....] pruebas.drfleming_zips 3.83MB/3.83MB (100.0%)
2026-02-26T09:28:11.835+0100 [#####.....] pruebas.drfleming_zips 3.83MB/3.83MB (100.0%)
2026-02-26T09:28:11.949+0100 [#####.....] pruebas.drfleming_zips 3.83MB/3.83MB (100.0%)
2026-02-26T09:28:11.949+0100 29471 document(s) imported successfully. 0 document(s) failed to import.
```



3.) Realiza una consulta y comprueba que los datos se han importado correctamente.

```
> use pruebas
< switched to db pruebas
> db.drfleming_zips.find().limit(5)
< {
  _id: ObjectId('5c8eccc1caa187d17ca6ed1a'),
  city: 'HUEYTOWN',
  zip: '35023',
  loc: {
    y: 33.414625,
    x: 86.999607
  },
  pop: 39677,
  state: 'AL'
}
{
  _id: ObjectId('5c8eccc1caa187d17ca6ed22'),
  city: 'CHELSEA',
  zip: '35043',
  loc: {
    y: 33.371582,
    x: 86.614132
  },
  pop: 4781,
  state: 'AL'
}
```

```
> db.drfleming_zips.countDocuments()
< 29471
```

4.) Exporta la base de datos de sample_trainings y muestra el resultado.

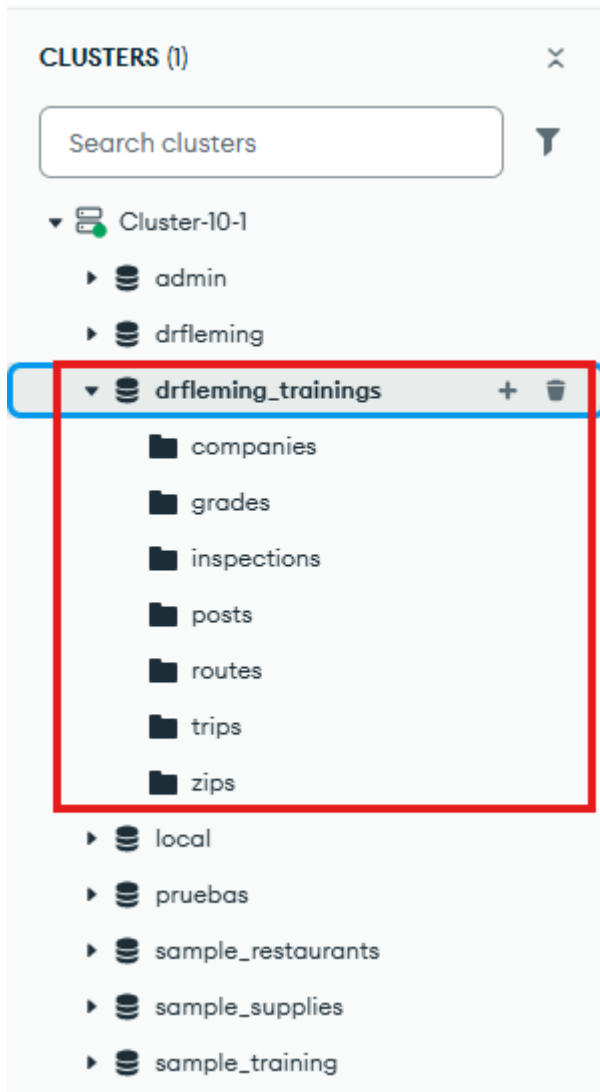
```
C:\MongoDB-Tools\bin>mongodump --uri="mongodb+srv://admin:admin@cluster-10-1.fegquti.mongodb.net/" --db sample_training
--out backup/ --numParallelCollections=1
2026-02-26T09:43:04.509+0100   writing sample_training.companies to backup/sample_training/companies.bson
2026-02-26T09:43:07.124+0100   [.....] sample_training.companies 0/9500 (0.0%)
2026-02-26T09:43:10.125+0100   [.....] sample_training.companies 0/9500 (0.0%)
2026-02-26T09:43:13.125+0100   [.....] sample_training.companies 0/9500 (0.0%)
2026-02-26T09:43:16.127+0100   [.....] sample_training.companies 0/9500 (0.0%)

2026-02-26T10:12:43.125+0100   [###.....] sample_training.posts 101/500 (20.2%)
2026-02-26T10:12:46.124+0100   [###.....] sample_training.posts 101/500 (20.2%)
2026-02-26T10:12:48.180+0100   [#####] sample_training.posts 500/500 (100.0%)
2026-02-26T10:12:48.180+0100   done dumping sample_training.posts (500 documents)
```

MongoDB-Tools > bin > backup > sample_training				
Ordenar Ver ...				
Nombre	Fecha de modificación	Tipo	Tamaño	
companies.bson	26/02/2026 9:54	Archivo BSON	35.621 KB	
companies.metadata.json	26/02/2026 9:43	Archivo de origen ...	1 KB	
grades.bson	26/02/2026 10:02	Archivo BSON	22.754 KB	
grades.metadata.json	26/02/2026 9:43	Archivo de origen ...	1 KB	
inspections.bson	26/02/2026 10:08	Archivo BSON	21.635 KB	
inspections.metadata.json	26/02/2026 9:43	Archivo de origen ...	1 KB	
posts.bson	26/02/2026 10:12	Archivo BSON	17.045 KB	
posts.metadata.json	26/02/2026 9:43	Archivo de origen ...	1 KB	
prelude.json	26/02/2026 10:12	Archivo de origen ...	1 KB	
routes.bson	26/02/2026 9:57	Archivo BSON	12.045 KB	
routes.metadata.json	26/02/2026 9:43	Archivo de origen ...	1 KB	
trips.bson	26/02/2026 10:03	Archivo BSON	4.184 KB	
trips.metadata.json	26/02/2026 9:43	Archivo de origen ...	1 KB	
zips.bson	26/02/2026 10:08	Archivo BSON	3.209 KB	
zips.metadata.json	26/02/2026 9:43	Archivo de origen ...	1 KB	

5.) Importa la base de datos recién exportada en drfleming_trainings

```
C:\MongoDB-Tools\bin>"C:\MongoDB-Tools\bin\mongorestore.exe" --uri="mongodb+srv://admin:admin@cluster-10-1.fegquti.mongodb.net/" --nsFrom="sample_training.*" --nsTo="drfleming_trainings.*" "backup/"
2026-03-02T09:00:00.295+0100 preparing collections to restore from
2026-03-02T09:00:00.316+0100 don't know what to do with file "backup\sample_training\prelude.json", skipping...
2026-03-02T09:00:00.325+0100 reading metadata for drfleming_trainings.grades from backup\sample_training\grades.metad
ata.json
2026-03-02T09:00:00.330+0100 reading metadata for drfleming_trainings.inspections from backup\sample_training\inspect
ions.metadata.json
2026-03-02T09:00:00.336+0100 reading metadata for drfleming_trainings.posts from backup\sample_training\posts.metadat
a.json
2026-03-02T09:00:00.342+0100 reading metadata for drfleming_trainings.routes from backup\sample_training\routes.metad
ata.json
2026-03-02T09:00:00.347+0100 reading metadata for drfleming_trainings.trips from backup\sample_training\trips.metadat
a.json
2026-03-02T09:00:00.351+0100 reading metadata for drfleming_trainings.zips from backup\sample_training\zips.metadata.
json
2026-03-02T09:00:00.357+0100 reading metadata for drfleming_trainings.companies from backup\sample_training\companies
.metadata.json
```



6.) Realiza una consulta y comprueba que los datos se han importado correctamente.

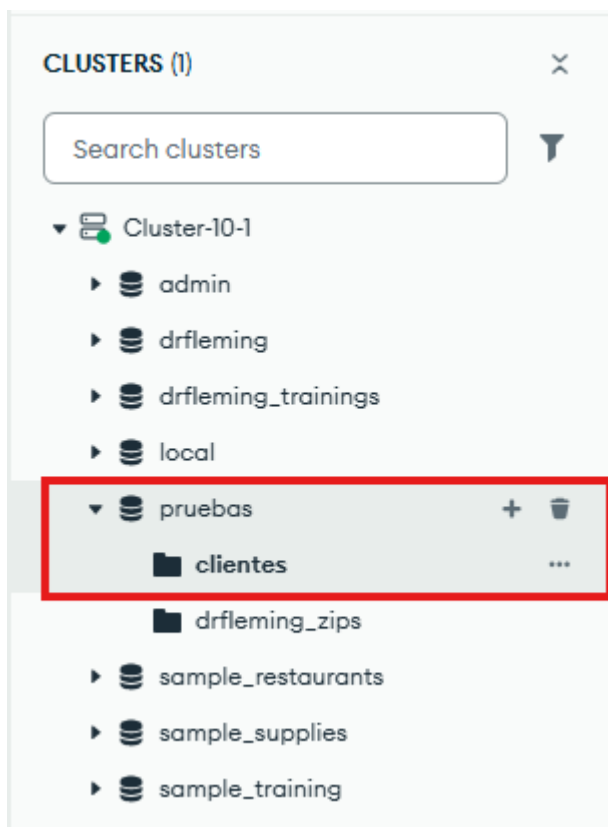
```
>_MONGOSH
> use drfleming_trainings
< switched to db drfleming_trainings
> show collections
< companies
  grades
  inspections
  posts
  routes
  trips
  zips
```



```
> db.zips.countDocuments()  
< 29471
```

7.) Importa el archivo clientes.csv dentro de la base de datos pruebas en la colección clientes.

```
C:\MongoDB-Tools\bin>"C:\MongoDB-Tools\bin\mongoimport.exe" --uri="mongodb+srv://admin:admin@cluster-10-1.fegquti.mongodb.net/" --db pruebas --collection clientes --type csv --headerline --file "C:\Users\Mañana\Downloads\clientes.csv"  
connected to: mongodb+srv://[**REDACTED**]@cluster-10-1.fegquti.mongodb.net/  
2026-03-02T09:21:59.025+0100 [#####] pruebas.clientes 56.0KB/337KB (16.6%)  
2026-03-02T09:22:02.025+0100 [#####] pruebas.clientes 84.0KB/337KB (24.9%)  
2026-03-02T09:22:05.025+0100 [#####] pruebas.clientes 108KB/337KB (32.1%)  
2026-03-02T09:22:11.025+0100 [#####] pruebas.clientes 136KB/337KB (40.4%)  
2026-03-02T09:22:14.025+0100 [#####] pruebas.clientes 164KB/337KB (48.7%)  
2026-03-02T09:22:17.025+0100 [#####] pruebas.clientes 192KB/337KB (57.0%)  
2026-03-02T09:22:20.025+0100 [#####] pruebas.clientes 244KB/337KB (72.5%)  
2026-03-02T09:22:23.026+0100 [#####] pruebas.clientes 328KB/337KB (97.4%)  
2026-03-02T09:22:24.134+0100 [#####] pruebas.clientes 337KB/337KB (100.0%)  
2026-03-02T09:22:24.135+0100 12434 document(s) imported successfully. 0 document(s) failed to import.
```



8.) Realiza una consulta y comprueba que los datos se han importado correctamente.

```
> use pruebas
< switched to db pruebas
> db.clientes.find().limit(5)
< {
  '1': 9,
  _id: ObjectId('69a548a70f6636a83c9684b1'),
  Richard: 'Mary',
  Hernandez: 'Perez',
  Brownsville: 'Caguas'
}
{
  '1': 2,
  _id: ObjectId('69a548a70f6636a83c9684b2'),
  Richard: 'Mary',
  Hernandez: 'Barrett',
  Brownsville: 'Littleton'
}
```

```
> db.clientes.countDocuments()
< 12434
```

9.) Importa el archivo poblacion.tsv dentro de la base de datos pruebas en la colección poblacion.

```
C:\MongoDB-Tools\bin>"C:\MongoDB-Tools\bin\mongoimport.exe" --uri="mongodb+srv://admin:admin@cluster-10-1.fegquti.mongodb.net/" --db pruebas --collection poblacion --type tsv --headerline --file "C:\Users\Mañana\Downloads\poblacion.tsv"
2026-03-02T09:25:17.767+0100 connected to: mongodb+srv://[**REDACTED**]@cluster-10-1.fegquti.mongodb.net/
2026-03-02T09:25:20.767+0100 [#####] pruebas.poblacion 48.0KB/342KB (14.0%)
2026-03-02T09:25:23.767+0100 [#####] pruebas.poblacion 88.0KB/342KB (25.7%)
2026-03-02T09:25:26.768+0100 [#####] pruebas.poblacion 132KB/342KB (38.5%)
2026-03-02T09:25:29.767+0100 [#####] pruebas.poblacion 212KB/342KB (61.9%)
2026-03-02T09:25:32.767+0100 [#####] pruebas.poblacion 256KB/342KB (74.7%)
2026-03-02T09:25:35.767+0100 [#####] pruebas.poblacion 296KB/342KB (86.4%)
2026-03-02T09:25:38.768+0100 [#####] pruebas.poblacion 340KB/342KB (99.3%)
2026-03-02T09:25:41.767+0100 [#####] pruebas.poblacion 342KB/342KB (100.0%)
2026-03-02T09:25:42.038+0100 [#####] pruebas.poblacion 342KB/342KB (100.0%)
2026-03-02T09:25:42.038+0100 8116 document(s) imported successfully. 0 document(s) failed to import.
```

CLUSTERS (1)

Search clusters

▼ Cluster-10-1

▶ admin

▶ drfleming

▶ drfleming_trainings

▶ local

▼ pruebas

clientes

drfleming_zips

poblacion

▶ sample_restaurants

▶ sample_supplies

▶ sample_training

10.) Realiza una consulta y comprueba que los datos se han importado correctamente.

```
> db.poblacion.countDocuments()
< 8116
> db.poblacion.find().limit(5)
< {
  _id: ObjectId('69a5496dc006db173fea165b'),
  CPRO: 2,
  PROVINCIA: 'Albacete',
  CMUN: 1,
  NOMBRE: 'Abengibre',
  POB13: 819,
  HOMBRES: 409,
  MUJERES: 410
}
{
  _id: ObjectId('69a5496dc006db173fea165c'),
  CPRO: 2,
  PROVINCIA: 'Albacete',
  CMUN: 2,
  NOMBRE: 'Alatoz',
  POB13: 572,
  HOMBRES: 318,
  MUJERES: 254
}
```

Apartado E

Escribe los comandos necesarios para realizar las siguientes consultas agregadas sobre la colección `sample_supplies.sales`:

1.) Cantidad total de ventas registradas.

```
> db.sales.aggregate([
  { $count: "totalVentas" }
])
< {
  totalVentas: 5000
}
```

2.) Cantidad total de ventas por método de compra (purchaseMethod).

```
> db.sales.aggregate([
  {
    $group: {
      _id: "$purchaseMethod",
      total: { $sum: 1 }
    }
  }
])
< {
  _id: 'Phone',
  total: 596
}
{
  _id: 'In store',
  total: 2819
}
{
  _id: 'Online',
  total: 1585
}
```

3.) Investiga el operador \$unwind y calcula el ingreso total de cada producto (name dentro de items)

```
> db.sales.aggregate([
  { $unwind: "$items" },
  {
    $group: {
      _id: "$items.name",
      ingresoTotal: {
        $sum: {
          $multiply: [
            { $toDouble: "$items.price" },
            "$items.quantity"
          ]
        }
      }
    }
  }
])
< {
  _id: 'printer paper',
  ingresoTotal: 367459.29
}
{
  _id: 'laptop',
  ingresoTotal: 6775977.07
}
{
  _id: 'pens',
  ingresoTotal: 581843.27
}
```

4.) Calcular el promedio de elementos por venta

```
> db.sales.aggregate([
  {
    $project: {
      numItems: { $size: "$items" }
    }
  },
  {
    $group: {
      _id: null,
      promedio: { $avg: "$numItems" }
    }
  }
])
< {
  _id: null,
  promedio: 5.4876
}
```

5.) A partir de todos los elementos, calcula el ingreso total por ventas, teniendo en cuenta el precio y la cantidad de cada elemento.

```
> db.sales.aggregate([
  { $unwind: "$items" },
  {
    $group: {
      _id: null,
      ingresoTotal: {
        $sum: {
          $multiply: [
            { $toDouble: "$items.price" },
            "$items.quantity"
          ]
        }
      }
    }
  }
])
< {
  _id: null,
  ingresoTotal: 9894572.27
}
```


6.) Encontrar los 3 productos más vendidos (por cantidad de unidades vendidas).

```
> db.sales.aggregate([
  { $unwind: "$items" },
  {
    $group: {
      _id: "$items.name",
      totalUnidades: { $sum: "$items.quantity" }
    }
  },
  { $sort: { totalUnidades: -1 } },
  { $limit: 3 }
])
< {
  _id: 'binder',
  totalUnidades: 25493
}
{
  _id: 'envelopes',
  totalUnidades: 25078
}
{
  _id: 'notepad',
  totalUnidades: 20727
}
```

7.) Averiguar cuantas ventas se han realizado del producto notepad.

```
> db.sales.aggregate([
  { $match: { "items.name": "notepad" } },
  { $count: "ventasConNotepad" }
])
< {
  ventasConNotepad: 3884
}
```

8.) Averiguar cuantos notepad se han vendido y cuanto se ha recaudado por sus ventas.

```
> db.sales.aggregate([
  { $unwind: "$items" },
  { $match: { "items.name": "notepad" } },
  {
    $group: {
      _id: null,
      totalUnidades: { $sum: "$items.quantity" },
      totalRecaudado: {
        $sum: {
          $multiply: [
            { $toDouble: "$items.price" },
            "$items.quantity"
          ]
        }
      }
    }
  }
])
< {
  _id: null,
  totalUnidades: 20727,
  totalRecaudado: 463615.48
}
```

9.) Averiguar los tres productos con un precio medio más alto.

```
> db.sales.aggregate([
  { $unwind: "$items" },
  {
    $group: {
      _id: "$items.name",
      precioMedio: { $avg: { $toDouble: "$items.price" } }
    }
  },
  { $sort: { precioMedio: -1 } },
  { $limit: 3 }
])
< {
  _id: 'laptop',
  precioMedio: 992.1520079260238
}
{
  _id: 'backpack',
  precioMedio: 118.40834127327847
}
{
  _id: 'pens',
  precioMedio: 42.23900636104409
}
```

10.) Averiguar la cantidad de hombres y mujeres que han realizado compras, así como sus edades medias.

```
> db.sales.aggregate([
  {
    $group: {
      _id: "$customer.gender",
      totalClientes: { $sum: 1 },
      edadMedia: { $avg: "$customer.age" }
    }
  }
])
< {
  _id: 'M',
  totalClientes: 2473,
  edadMedia: 44.44965628790942
}
{
  _id: 'F',
  totalClientes: 2527,
  edadMedia: 45.01543332014246
}
```

11.) Recuperar el nombre y el email de los 5 clientes que más han gastado.

```
> db.sales.aggregate([
  { $unwind: "$items" },
  {
    $group: {
      _id: {
        email: "$customer.email",
        name: "$customer.gender"
      },
      totalGastado: {
        $sum: {
          $multiply: [
            { $toDouble: "$items.price" },
            "$items.quantity"
          ]
        }
      }
    }
  },
  { $sort: { totalGastado: -1 } },
  { $limit: 5 }
])
< {
  _id: {
    email: 'kegzuh@bok.hk',
    name: 'F'
  },
  totalGastado: 9385.19
}
{
  _id: {
    email: 'mebni@juubku.mo',
    name: 'F'
  },
  totalGastado: 9105.130000000001
}
```

12.) Para cada email, mostrar la cantidad de ventas realizadas, así como la cantidad de productos adquiridos.

```
> db.sales.aggregate([
  { $unwind: "$items" },
  {
    $group: {
      _id: "$customer.email",
      totalVentas: { $addToSet: "$_id" },
      totalProductos: { $sum: "$items.quantity" }
    }
  },
  {
    $project: {
      ventasRealizadas: { $size: "$totalVentas" },
      totalProductos: 1
    }
  }
])
< {
  _id: 'su@uhuwki.tt',
  totalProductos: 8,
  ventasRealizadas: 1
}
{
  _id: 'ribfo@ewves.ws',
  totalProductos: 28,
  ventasRealizadas: 1
}
```

13.) Recupera cual es el producto que más se ha vendido en London.

```
> db.sales.aggregate([
  { $match: { storeLocation: "London" } },
  { $unwind: "$items" },
  {
    $group: {
      _id: "$items.name",
      total: { $sum: "$items.quantity" }
    }
  },
  { $sort: { total: -1 } },
  { $limit: 1 }
])
< {
  _id: 'envelopes',
  total: 4037
}
```

14.) Obtener el nombre y el email del cliente, y el total gastado, pero solo para aquellos clientes que han gastado más de 1000\$.

```
> db.sales.aggregate([
  { $unwind: "$items" },
  {
    $group: {
      _id: {
        name: "$customer.gender",
        email: "$customer.email"
      },
      totalGastado: {
        $sum: {
          $multiply: [
            { $toDouble: "$items.price" },
            "$items.quantity"
          ]
        }
      }
    }
  },
  { $match: { totalGastado: { $gt: 1000 } } }
])
< {
  _id: {
    name: 'M',
    email: 'fataru@ere.pw'
  },
  totalGastado: 1679.7
}
{
  _id: {
    name: 'M',
    email: 'seledfeb@dirpiwsi.sb'
  },
  totalGastado: 2091.34
}
```


15.) Averiguar cuáles han sido los tres años que más ventas se han realizado.

```
> db.sales.aggregate([
  {
    $group: {
      _id: { $year: "$saleDate" },
      totalVentas: { $sum: 1 }
    }
  },
  { $sort: { totalVentas: -1 } },
  { $limit: 3 }
])
< {
  _id: 2017,
  totalVentas: 1052
}
{
  _id: 2016,
  totalVentas: 1010
}
{
  _id: 2015,
  totalVentas: 1006
}
```

Apartado F

Escribe los comandos necesarios para realizar las siguientes consultas agregadas sobre la colección `sample_restaurants.restaurants`:

1.) Cantidad total de restaurantes.

```
> db.restaurants.aggregate([
  { $count: "totalRestaurantes" }
])
< {
  totalRestaurantes: 25360
}
```

2.) Cantidad de restaurantes por barrio (propiedad borough).

```
> db.restaurants.aggregate([
  {
    $group: {
      _id: "$borough",
      total: { $sum: 1 }
    }
  }
])
< {
  _id: 'Bronx',
  total: 2338
}
{
  _id: 'Brooklyn',
  total: 6086
}
{
  _id: 'Queens',
  total: 5656
}
{
  _id: 'La Calzada',
  total: 1
}
```

3.) Recupera los cinco barrios con mayor cantidad de restaurantes.

```
> db.restaurants.aggregate([
  {
    $group: {
      _id: "$borough",
      total: { $sum: 1 }
    }
  },
  { $sort: { total: -1 } },
  { $limit: 5 }
])
< {
  _id: 'Manhattan',
  total: 10259
}
{
  _id: 'Brooklyn',
  total: 6086
}
{
  _id: 'Queens',
  total: 5656
}
```

4.) Investiga el operador \$unwind, y recupera la calificación media del restaurante Palm Restauran.

```
> db.restaurants.aggregate([
  { $match: { name: "Palm Restaurant" } },
  { $unwind: "$grades" },
  {
    $group: {
      _id: "$name",
      mediaScore: { $avg: "$grades.score" }
    }
  }
])
< {
  _id: 'Palm Restaurant',
  mediaScore: 11.133333333333333
}
```

5.) Recupera la fecha de la última calificación de cada restaurante.

```
> db.restaurants.aggregate([
  { $unwind: "$grades" },
  {
    $group: {
      _id: "$name",
      ultimaFecha: { $max: "$grades.date" }
    }
  }
])
< {
  _id: 'Corwn Fried Chicken And Fish',
  ultimaFecha: 2014-05-05T00:00:00.000Z
}
{
  _id: 'Good To Go',
  ultimaFecha: 2014-10-16T00:00:00.000Z
}
{
  _id: 'T2 - Go',
  ultimaFecha: 2014-11-26T00:00:00.000Z
}
{
  _id: 'Maddens Pub',
  ultimaFecha: 2014-10-23T00:00:00.000Z
}
```

6.) Encuentra todos los restaurantes que al menos tienen una calificación de A.

```
> db.restaurants.aggregate([
  { $match: { "grades.grade": "A" } },
  {
    $project: {
      _id: 0,
      name: 1,
      borough: 1
    }
  }
])
< {
  borough: 'Bronx',
  name: 'Morris Park Bake Shop'
}
{
  borough: 'Queens',
  name: 'Tov Kosher Kitchen'
}
{
  borough: 'Queens',
  name: 'Brunos On The Boulevard'
}
{
  borough: 'Brooklyn',
  name: 'C & C Catering Service'
}
```

7.) Mediante \$unwind, cuenta cuántas calificaciones de tipo A, B, C, etc.... existen en total en todos los restaurantes.

```
> db.restaurants.aggregate([
  { $unwind: "$grades" },
  {
    $group: {
      _id: "$grades.grade",
      total: { $sum: 1 }
    }
  }
])
< {
  _id: 'Z',
  total: 1337
}
{
  _id: 'B',
  total: 12603
}
{
  _id: 'C',
  total: 3145
}
{
  _id: 'P',
  total: 1197
}
```


8.) Mostrar los restaurantes que tienen más de tres calificaciones.

```
> db.restaurants.aggregate([
  {
    $project: {
      name: 1,
      totalGrades: { $size: "$grades" }
    }
  },
  { $match: { totalGrades: { $gt: 3 } } }
])
< {
  _id: ObjectId('5eb3d668b31de5d588f4292e'),
  name: 'Morris Park Bake Shop',
  totalGrades: 5
}
{
  _id: ObjectId('5eb3d668b31de5d588f4292f'),
  name: 'Tov Kosher Kitchen',
  totalGrades: 4
}
{
  _id: ObjectId('5eb3d668b31de5d588f42930'),
  name: 'Brunos On The Boulevard',
  totalGrades: 4
}
{
  _id: ObjectId('5eb3d668b31de5d588f42934'),
  name: 'C & C Catering Service',
  totalGrades: 4
}
```

9.) Mostrar los restaurantes que tienen más de cuatro calificaciones y que hagan cocina American.

```
> db.restaurants.aggregate([
  {
    $project: {
      name: 1,
      cuisine: 1,
      totalGrades: { $size: "$grades" }
    }
  },
  {
    $match: {
      totalGrades: { $gt: 4 },
      cuisine: "American"
    }
  }
])
< {
  _id: ObjectId('5eb3d668b31de5d588f42931'),
  cuisine: 'American',
  name: 'Regina Caterers',
  totalGrades: 5
}
{
  _id: ObjectId('5eb3d668b31de5d588f42940'),
  cuisine: 'American',
  name: 'Glorious Food',
  totalGrades: 5
}
```

10.) Encontrar el restaurante con la calificación A más alta (mayor score).

```
> db.restaurants.aggregate([
  { $unwind: "$grades" },
  { $match: { "grades.grade": "A" } },
  { $sort: { "grades.score": -1 } },
  { $limit: 1 },
  {
    $project: {
      name: 1,
      score: "$grades.score"
    }
  }
])
< {
  _id: ObjectId('5eb3d668b31de5d588f42eec'),
  name: "Gemini'S Lounge",
  score: 27
}
```

11.) Calcular la diferencia entre el score más alto y bajo para cada restaurante que tenga al menos dos grades.

```
> db.restaurants.aggregate([
  {
    $project: {
      name: 1,
      totalGrades: { $size: "$grades" },
      maxScore: { $max: "$grades.score" },
      minScore: { $min: "$grades.score" }
    }
  },
  {
    $match: { totalGrades: { $gte: 2 } }
  },
  {
    $project: {
      name: 1,
      diferencia: { $subtract: ["$maxScore", "$minScore"] }
    }
  }
])
< {
  _id: ObjectId('5eb3d668b31de5d588f4292e'),
  name: 'Morris Park Bake Shop',
  diferencia: 12
}
{
  _id: ObjectId('5eb3d668b31de5d588f4292f'),
  name: 'Tov Kosher Kitchen',
  diferencia: 12
}
```

12.) Obtener el nombre del restaurante, su dirección (street y zipcode) y la última fecha de inspección (grade.date) para los restaurantes del Bronx.

```
> db.restaurants.aggregate([
  { $match: { borough: "Bronx" } },
  { $unwind: "$grades" },
  {
    $group: {
      _id: "$_id",
      name: { $first: "$name" },
      street: { $first: "$address.street" },
      zipcode: { $first: "$address.zipcode" },
      ultimaFecha: { $max: "$grades.date" }
    }
  }
])
< {
  _id: ObjectId('5eb3d669b31de5d588f481b6'),
  name: "Jacqueline'S Bakery Restaurant",
  street: 'Westchester Avenue',
  zipcode: '10472',
  ultimaFecha: 2014-06-25T00:00:00.000Z
}
{
  _id: ObjectId('5eb3d668b31de5d588f42a86'),
  name: 'Yankee Jz Pizza',
  street: 'Westchester Avenue',
  zipcode: '10472',
  ultimaFecha: 2015-01-13T00:00:00.000Z
}
```

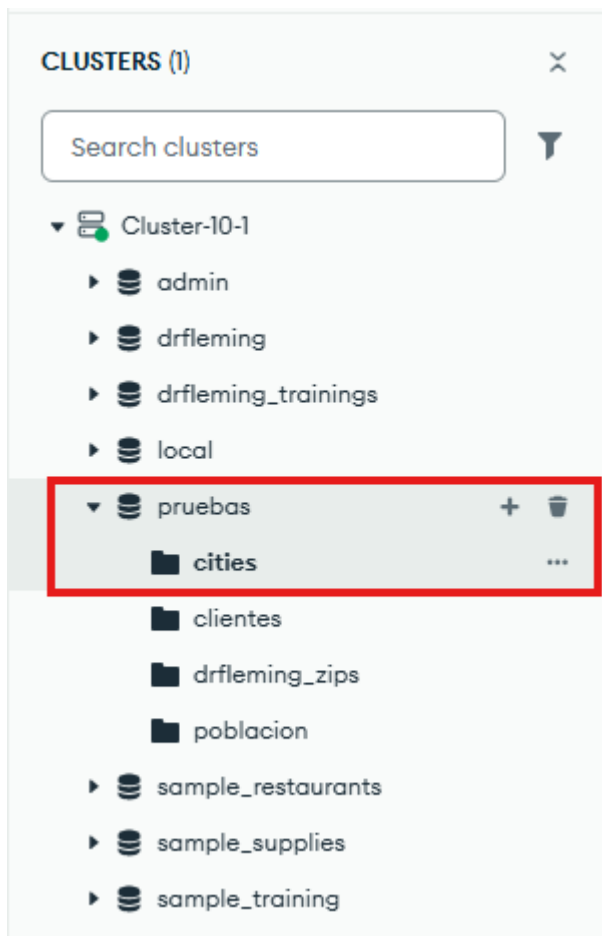
13.) Encontrar los 3 barrios tienen la mayor variedad de tipos de cocina.

```
> db.restaurants.aggregate([
  {
    $group: {
      _id: "$borough",
      tiposCocina: { $addToSet: "$cuisine" }
    }
  },
  {
    $project: {
      variedad: { $size: "$tiposCocina" }
    }
  },
  { $sort: { variedad: -1 } },
  { $limit: 3 }
])
< {
  _id: 'Manhattan',
  variedad: 82
}
{
  _id: 'Brooklyn',
  variedad: 80
}
{
  _id: 'Queens',
  variedad: 74
}
```

Apartado G

Importa el fichero mongo_cities1000.json en pruebas.cities.
Sobre dichos datos, resuelve las siguientes consultas:

```
C:\Users\Mañana>"C:\MongoDB-Tools\bin\mongoimport.exe" --uri="mongodb+srv://admin:admin@cluster-10-1.fegquti.mongodb.net/" --db pruebas --collection cities --type json --file "C:\Users\Mañana\Downloads\mongo_cities1000.json"
2026-03-03T09:03:21.072+0100 connected to: mongodb+srv://[REDACTED]@cluster-10-1.fegquti.mongodb.net/
2026-03-03T09:03:24.072+0100 [.....] pruebas.cities 152KB/13.7MB (1.1%)
2026-03-03T09:03:27.072+0100 [.....] pruebas.cities 296KB/13.7MB (2.1%)
2026-03-03T09:03:30.072+0100 [.....] pruebas.cities 437KB/13.7MB (3.1%)
2026-03-03T09:03:33.072+0100 [.....] pruebas.cities 576KB/13.7MB (4.1%)
2026-03-03T09:03:36.072+0100 [#.....] pruebas.cities 723KB/13.7MB (5.2%)
2026-03-03T09:03:39.072+0100 [#.....] pruebas.cities 869KB/13.7MB (6.2%)
2026-03-03T09:03:42.072+0100 [#.....] pruebas.cities 1013KB/13.7MB (7.2%)
```



1.) Calcula el número total de ciudades.

```
> db.cities.aggregate([
  { $count: "totalCiudades" }
])
< {
  totalCiudades: 80159
}
```

2.) Recupera todos los datos de la ciudad cuyo nombre es Oviedo.

```
> db.cities.find({ name: "Oviedo" })
< {
  _id: ObjectId('69a6960b2333bc120cbd644d'),
  name: 'Oviedo',
  country: 'DO',
  timezone: 'America/Santo_Domingo',
  population: 2671,
  location: {
    latitude: 17.78333,
    longitude: -71.36667
  }
}
{
  _id: ObjectId('69a6960d2333bc120cbd6e23'),
  name: 'Oviedo',
  country: 'ES',
  timezone: 'Europe/Madrid',
  population: 224005,
  location: {
    latitude: 43.36029,
    longitude: -5.84476
  }
}
```

3.) Recupera únicamente la población de Vergel.

```
> db.cities.find(
  { name: "Vergel" },
  { _id: 0, population: 1 }
)
< {
  population: 4372
}
```

4.) Calcula el número de ciudades de España.

```
> db.cities.countDocuments({ country: "ES" })
< 2724
```


5.) Datos de las ciudades españolas con más de un millón de habitantes.

```
> db.cities.find({
  country: "ES",
  population: { $gt: 1000000 }
})
< {
  _id: ObjectId('69a6960d2333bc120cbd6ebc'),
  name: 'Madrid',
  country: 'ES',
  timezone: 'Europe/Madrid',
  population: 3255944,
  location: {
    latitude: 40.4165,
    longitude: -3.70256
  }
}
{
  _id: ObjectId('69a696112333bc120cbd70df'),
  name: 'Barcelona',
  country: 'ES',
  timezone: 'Europe/Madrid',
  population: 1621537,
  location: {
    longitude: 41.38879,
    latitude: 2.15899
  }
}
```

6.) Recupera la cantidad de ciudades que hay entre Andorra (AD) y España.

```
> db.cities.countDocuments({
  country: { $in: ["AD", "ES"] }
})
< 2733
```

7.) Listado con el nombre y la población de las 10 ciudades más pobladas.

```
> db.cities.aggregate([
  { $sort: { population: -1 } },
  { $limit: 10 },
  { $project: { _id: 0, name: 1, population: 1 } }
])
< {
  name: 'Shanghai',
  population: 14608512
}
{
  name: 'Buenos Aires',
  population: 13076300
}
{
  name: 'Mumbai',
  population: 12691836
}
{
  name: 'Mexico City',
  population: 12294193
}
```

8.) Nombre de las distintas zonas horarias en España.

```
> db.cities.distinct("timezone", { country: "ES" })
< [ 'Africa/Ceuta', 'Atlantic/Canary', 'Europe/Madrid' ]
```

9.) Ciudades españolas que su zona horaria no sea Europe/Madrid.

```
> db.cities.distinct("timezone", { country: "ES" })
< [ 'Africa/Ceuta', 'Atlantic/Canary', 'Europe/Madrid' ]
> db.cities.find({
  country: "ES",
  timezone: { $ne: "Europe/Madrid" }
})
< {
  _id: ObjectId('69a6960b2333bc120cbd6710'),
  name: 'Yaiza',
  country: 'ES',
  timezone: 'Atlantic/Canary',
  population: 8777,
  location: {
    latitude: 28.95678,
    longitude: -13.76535
  }
}
{
  _id: ObjectId('69a6960b2333bc120cbd6750'),
  name: 'Vilaflor',
  country: 'ES',
  timezone: 'Atlantic/Canary',
  population: 1981,
  location: {
    latitude: 28.15,
    longitude: -16.63333
  }
}
```

10.) Mostrar las ciudades españolas que comiencen por Ben

```
> db.cities.find({
  country: "ES",
  name: { $regex: "^Ben" }
})
< {
  _id: ObjectId('69a6960c2333bc120cbd6b3c'),
  name: 'Benitachell',
  country: 'ES',
  timezone: 'Europe/Madrid',
  population: 3630,
  location: {
    longitude: 38.73333,
    latitude: 0.15
  }
}
{
  _id: ObjectId('69a6960c2333bc120cbd6b3d'),
  name: 'Benissa',
  country: 'ES',
  timezone: 'Europe/Madrid',
  population: 13221,
  location: {
    longitude: 38.71492,
    latitude: 0.04849
  }
}
```

11.) Devolver las ciudades que su zona horaria sea Atlantic/Canary o Africa/Ceuta.

```
> db.cities.find({
  timezone: { $in: ["Atlantic/Canary", "Africa/Ceuta"] }
})
< {
  _id: ObjectId('69a6960b2333bc120cbd6710'),
  name: 'Yaiza',
  country: 'ES',
  timezone: 'Atlantic/Canary',
  population: 8777,
  location: {
    latitude: 28.95678,
    longitude: -13.76535
  }
}
{
  _id: ObjectId('69a6960b2333bc120cbd6750'),
  name: 'Vilaflor',
  country: 'ES',
  timezone: 'Atlantic/Canary',
  population: 1981,
  location: {
    latitude: 28.15,
    longitude: -16.63333
  }
}
```

12.) Obtén el nombre y la población de las tres ciudades europeas más pobladas

```
> db.cities.aggregate([
  {
    $match: {
      country: {
        $in: ["ES","FR","IT","DE","PT","AD","RU","UK","NL","BE","CH","AT","PL"]
      }
    }
  },
  { $sort: { population: -1 } },
  { $limit: 3 },
  {
    $project: {
      _id: 0,
      name: 1,
      population: 1,
      country: 1
    }
  }
])
< {
  name: 'Moscow',
  country: 'RU',
  population: 10381222
}
{
  name: 'Saint Petersburg',
  country: 'RU',
  population: 4039745
}
{
  name: 'Berlin',
  country: 'DE',
  population: 3426354
}
```

13.) Calcula la cantidad de ciudades españolas cuyas coordenadas de longitud estén comprendidas entre -0.1 y 0.1.

```
> db.cities.countDocuments({
  country: "ES",
  longitude: { $gte: -0.1, $lte: 0.1 }
})
< 0
```

14.) Mostrar los 3 países con mayor número de ciudades.

```
> db.cities.aggregate([
  {
    $group: {
      _id: "$country",
      total: { $sum: 1 }
    }
  },
  { $sort: { total: -1 } },
  { $limit: 3 }
])
< {
  _id: 'FR',
  total: 8557
}
{
  _id: 'DE',
  total: 6935
}
{
  _id: 'IT',
  total: 6192
}
```

15.) Devolver los 3 países con mayor número de habitantes.

```
> db.cities.aggregate([
  {
    $group: {
      _id: "$country",
      poblacionTotal: { $sum: "$population" }
    }
  },
  { $sort: { poblacionTotal: -1 } },
  { $limit: 3 }
])
< {
  _id: 'CN',
  poblacionTotal: 282839031
}
{
  _id: 'IN',
  poblacionTotal: 272149640
}
{
  _id: 'BR',
  poblacionTotal: 132102085
}
```


16.) Calcular y devolver la población total de España, Francia e Italia, en orden descendente.

```
> db.cities.aggregate([
  {
    $match: { country: { $in: ["ES", "FR", "IT"] } }
  },
  {
    $group: {
      _id: "$country",
      total: { $sum: "$population" }
    }
  },
  { $sort: { total: -1 } }
])
< {
  _id: 'IT',
  total: 56815974
}
{
  _id: 'FR',
  total: 51573677
}
{
  _id: 'ES',
  total: 44300931
}
```

17.) Calcular la latitud y longitud media para cada zona horaria europea.

```
> db.cities.aggregate([
  {
    $match: {
      timezone: { $regex: "^Europe/" }
    }
  },
  {
    $group: {
      _id: "$timezone",
      latitud_media: { $avg: "$location.latitude" },
      longitud_media: { $avg: "$location.longitude" }
    }
  },
  {
    $project: {
      _id: 0,
      timezone: "$_id",
      latitud_media: 1,
      longitud_media: 1
    }
  }
])
< {
  latitud_media: 14.823260914634147,
  longitud_media: 46.194177743902436,
  timezone: 'Europe/Ljubljana'
}
{
  latitud_media: 39.92264576309795,
  longitud_media: -8.54203854214123,
  timezone: 'Europe/Lisbon'
}
```

18.) Devolver los países en la posición 5ª y 6ª si ordenamos su población total en orden descendente.

```
> db.cities.aggregate([
  {
    $group: {
      _id: "$country",
      total: { $sum: "$population" }
    }
  },
  { $sort: { total: -1 } },
  { $skip: 4 },
  { $limit: 2 }
])
< {
  _id: 'JP',
  total: 102632560
}
{
  _id: 'MX',
  total: 85555230
}
```

19.) Calcular las zonas horarias con mayor y menor número de habitantes para España y Rusia. Añade también un campo extra que indique cuántas zonas horarias tiene cada país.

```
> db.cities.aggregate([
  {
    $match: { country: { $in: ["ES", "RU"] } }
  },
  {
    $group: {
      _id: { country: "$country", timezone: "$timezone" },
      total: { $sum: "$population" }
    }
  },
  {
    $group: {
      _id: "$_id.country",
      zonas: { $push: { tz: "$_id.timezone", total: "$total" } },
      totalZonas: { $sum: 1 }
    }
  }
])
< {
  _id: 'RU',
  zonas: [
    {
      tz: 'Asia/Kamchatka',
      total: 274516
    },
    {
      tz: 'Europe/Moscow',
      total: 65316206
    },
  ],
}
```

20.) Calcular qué porcentaje de la población representa la ciudad más poblada de España.

```
> db.cities.aggregate([
  { $match: { country: "ES" } },
  {
    $group: {
      _id: null,
      totalPais: { $sum: "$population" },
      maxCiudad: { $max: "$population" }
    }
  },
  {
    $project: {
      porcentaje: {
        $multiply: [
          { $divide: ["$maxCiudad", "$totalPais"] },
          100
        ]
      }
    }
  }
])
< {
  _id: null,
  porcentaje: 7.3496062644823414
}
```

21.) Utilizando unwind, devuelve los pares únicos (país y timezone) con un campo que indique la suma total de población del país.

```
> db.cities.aggregate([
  {
    $group: {
      _id: { country: "$country", timezone: "$timezone" },
      totalPais: { $sum: "$population" }
    }
  }
])
< {
  _id: {
    country: 'BM',
    timezone: 'Atlantic/Bermuda'
  },
  totalPais: 2704
}
{
  _id: {
    country: 'LU',
    timezone: 'Europe/Luxembourg'
  },
  totalPais: 327830
}
{
  _id: {
    country: 'GH',
    timezone: 'Africa/Accra'
  },
  totalPais: 6968657
}
```

22.) Averigua qué nombres de ciudades se repiten entre distintos países.

```
> db.cities.aggregate([
  {
    $group: {
      _id: "$name",
      paises: { $addToSet: "$country" },
      total: { $sum: 1 }
    }
  },
  {
    $match: { total: { $gt: 1 } }
  }
])
< {
  _id: 'Gries',
  paises: [
    'FR',
    'DE'
  ],
  total: 2
}
{
  _id: 'Xinzhou',
  paises: [
    'CN'
  ],
  total: 3
}
```